CS 171 Assignment 6

Submission Instructions: Please submit the **.java files** to Canvas**.** Make sure **download Java template files** for each question before coding. Make sure that all Java files you submit can be compiled and run against Java Development Kit 8 (or higher). You can define a main method and test them before submit it.

Honor Code: Solve this assignment **individually**; do not collaborate with classmates or look for online solutions. We check for code plagiarism. The assignment is governed by the College Honor Code and Departmental Policy. Remember, **any code you submit must be your own;** otherwise, you risk being investigated by the Honor Council and facing the consequences of that. Finally, please remember to have the following comment included at the top of each of your .java file:

```
/*
THIS CODE WAS MY OWN WORK, IT WAS WRITTEN WITHOUT CONSULTING
CODE WRITTEN BY OTHER STUDENTS OR COPIED FROM ONLINE RESOURCES.
_Your_Name_Here_
*/
```

4. BST for Movie Search (50 pts) Congratulations! You've been hired by IMDB to implement a Binary Search Tree (BST) based index for indexing/searching movies efficiently in their IMDB dataset.

Requirements: Our goal is to quickly find the information associated with movies, sometimes based on a movie name (e.g. Gangster Squad), in other times based on the prefix of a name (e.g. Gangster*). In terms of **key-value** notations, your search **key** is the short or simplified title of a movie, and the associated **value** or data is the entire object of type ***MovieInfo***, which contains more information on the movie and is defined in the following class:

```
class MovieInfo {
   public String shortTitle; //simplified movie title, e.g. "Gangster Squad"
   public String fullTitle; //full title including year, e.g. "Gangster Squad
   (2012)" public int ID; //integer ID
}
```

The BST index is implemented as a stand-alone class ***BSTIndex***, with an inner class ***Node***. Each node within a ***BSTIndex*** tree contains 4 fields: key (of type String), data (of type *MovieInfo*), and left and right links to children nodes. The BST index tree should never contain duplicate nodes; i.e. each node key needs to be unique across the tree. You will complete the implementation of the following methods and operations in class ***BSTIndex***. All BST operations should be implemented using recursion.

1. ***public int calcHeight()*** This method should calculate and return the height of the current BSTIndex tree. We define the height of a tree (similar to our lecture slides) as the maximum depth in the tree plus 1. The height of a tree with only one node is 1, and the height of an empty tree is 0.

Important: Notice that this is a public method that calls another private method ***calcNodeHeight(Node t)*** and passes it a reference to the root. This is a common way of structuring BST methods such that external client code will not have direct access to the tree root. You will thus be implementing the code in the private ***calcNodeHeight(Node t)*** method. This applies to the rest of the methods in this question as well.

2. public void insertMovie (MovieInfo data) This method should insert the given data element into the proper place in the BST structure. If the input file turned out to have duplicate keys (i.e. repeated short movie titles), then your insertion method should detect that and **not** add the new duplicated movie to the tree (basically do nothing).
3. public MovieInfo findMovie (String key) This method should return the data element (i.e. the *MovieInfo* object) of the node where the movie's *shortTitle* matches the given key. Return *null* if the movie is not found.
4. public void printMoviesPrefix (String prefixStr) This method should print out all movies in the tree whose ***shortTitle*** begins with the passed prefix string. If no movies match the given prefix, nothing will be printed. The order of printing does not matter, but make sure to print each match in a separate line. The *IndexTester* class is provided which will test your *BSTIndex* class. The *IndexTester* creates an empty *BSTIndex* object then reads the data from an input movie file, builds a *MovieInfo* object for each row, and inserts it into the BST index (using the insert method you implemented in *BSTIndex* class). At this point, the BST index will be built for all the movie entries in the file. Then, the program will ask the user to enter a string. The code for parsing the input string and deciding which operation to invoke is part of the starter code.
Here's an example of how a program run might look like. Note that the data file name is passed as an argument to the program's *main*() method here using the command line:

```
> java IndexTester data/movies100K.txt Inserted 10000 records.
Inserted 20000 records.
Inserted 30000 records.
Inserted 40000 records.
Inserted 50000 records.
Inserted 60000 records.
Inserted 70000 records.
Inserted 80000 records.
Inserted 90000 records.
Inserted 100000 records.
Index building complete. Inserted 100000 records. Elapsed time = <your_result_ here>
seconds.
Welcome to the IMDB movie search engine!
- Enter search string to find a specific movie.
- Enter search string ending with `*' to print all movies that match this prefix -
Enter `h' to print tree height.
- Enter `q' to quit.
Enter Option: h
Height of BSTIndex tree = <your_result_here> Enter Option: Gang
Movie Gang Not Found
Enter Option: Gang*
[Finding all movies that start with Gang..] Gang tai Gang
Gangster Squad
Gang Wars
Gangotri
...
<rest_of_matches_here>
Enter Option: q
```

Getting started:

- Download and understand the starter code ***IndexTester.java, MovieInfo.java,*** and ***BSTIndex.java.***
- The input data is available on Canvas. Note that the full data file (movies.txt) is relatively large. You can copy the smaller extract (movies100K.txt) for debugging. They are all in the same format: one movie per line, with fields ID, shortTitle, and fullTitle,

separated by a tab character `\t'. Read IndexTester code to remind yourself how to read this data.

- Implement all missing code in ***BSTIndex.java***. This is the only file you are required to submit on Canvas for this question.
- Test your program with ***IndexTester*** on the available datasets, starting from the smaller one.

Starter code:

```
public class MovieInfo {
   String shortTitle;     // simplified movie title, e.g. "Gangster Squad"
   String fullTitle;     // full movie title including the year, e.g.
"Gangster Squad (2012)"
   int ID;                        // integer ID of the movie
   public MovieInfo(int id, String s, String f) {
      ID = id;
      shortTitle = s;
      fullTitle = f;
   }
}

public class IndexTester {

   public static void main( String[] args )
   {

      if(args.length < 1)
      {
         System.out.println("Error: IMDB file name missing! Usage: java
IndexTester data/fileNameHere.txt");
         return;
      }

      String fileName = args[0];
      int i = 0;
      long start = System.currentTimeMillis();
      BSTIndex bst = new BSTIndex();

      // Read the IMDB movies input file (e.g. "data/movies.txt" or
"data/movies100K.txt")
      try{
         Scanner scan = new Scanner(new FileInputStream(fileName));
         while(scan.hasNext()){
            String line = scan.nextLine();
            String [] fields = line.split("\t");
            int id = Integer.parseInt(fields[0].trim());
            String shortTitle = fields[1].trim();
            String fullTitle = fields[2].trim();
            MovieInfo info = new MovieInfo(id, shortTitle, fullTitle);

            bst.insertMovie(info); // insert movie into BST
            i++;
            if(i % 10000 == 0){
               System.out.println("Inserted " + i + " records.");
            }
         } // end of while loop
```

```
      } // end of try
      catch (FileNotFoundException e) {
        e.printStackTrace();
            System.out.println("\nFile " + fileName + " could not be read.
Are you sure you saved it in the right directory?");
            System.exit(-1); // exit program
    } // end of catch

      long end = System.currentTimeMillis();
      System.out.println("Index building complete. Inserted " + i + "
records. Elapsed time = " + (end - start)/1000 + " seconds.");

      Scanner input = new Scanner(System.in);
      System.out.println("\nWelcome to the IMDB movie search engine!");
      System.out.println("- Enter search string to find a specific movie. \n-
Enter search string ending with '*' to print all movies that match this
prefix. \n- Enter 'h' to print tree height. \n- Enter 'q' to quit.");

      System.out.print("Enter Option: ");
      while(input.hasNext()) {
         String search = input.nextLine().trim();

         if(search.equals("q")) break; // q = quit

         if(search.equals("h")){
            //compute and print the height of this tree
            System.out.println("Height of BSTIndex tree = " +
bst.calcHeight());
         }
         else if(search.indexOf('*') > 0)
         {
            //call prefix search
            System.out.println("[Finding all movies that start with " +
search.split("\\*")[0] + "..]");
            bst.printMoviesPrefix(search);
         }
         else
         {
            //call exact search to find a specific movie
            MovieInfo match = bst.findMovie(search);
            if(match == null)
                  System.out.println("Movie " + search + " Not Found");
            else
                  System.out.println(match.ID + " " + match.shortTitle + " "
+ match.fullTitle);
         }
         System.out.print("Enter Option: ");
      }
   }
}


public class BSTIndex {
      private class Node {
         private String key;
         private MovieInfo data;
```

```
        private Node left, right;

        public Node(MovieInfo info) {
            data = info;
            key = data.shortTitle;
        }
    }

    private Node root;

    public BSTIndex() {
        root = null;
    }

    // --------- [DO NOT MODIFY!] public BST methods  --------- //
    /* Important: Notice that each public method here calls another private
method while passing it a reference to the tree root. This is a common way of
structuring BST functions such that external client code will not have direct
access to the tree root. You will be implementing the code in the private
methods, not the public ones. */

    /* Calculate and return the height of the current tree. */
    public int calcHeight(){
        return calcNodeHeight(this.root);
    }

    /* Insert the given data element into the proper place in the BST
structure. */
    public void insertMovie(MovieInfo data) {
        root = insertMovie(data, this.root);
    }

    /* Find and return the data element (i.e. the MovieInfo object)
    of the node where the movie's shortTitle matches the given key.
    Return null if the movie is not found. */
    public MovieInfo findMovie(String key) {
        return findMovie(this.root, key);
    }

    /* Print out all movies in the database whose shortTitle begins with
    the passed prefix string. If no movies match the given prefix, nothing
    will be printed. The order of printing does not matter, but make sure
    to print each match in a separate line. */
    public void printMoviesPrefix(String prefix) {
        printMoviesPrefix(this.root, prefix);
    }
    // ----------------- end of public methods ------------------ //


    // ------------- [TODO] private BST methods --------------- //
    private int calcNodeHeight(Node t) {
        // ... TODO ... //
    }

    private Node insertMovie(MovieInfo data, Node t) {
        // ... TODO ... //
    }
```

```
	private MovieInfo findMovie(Node t, String key) {
		// ... TODO ... //
	}

	private void printMoviesPrefix(Node t, String prefix) {
		// ... TODO ... //
	}

}
```